

Activity 01

Compare Flutter, React Native, Kotlin Multiplatform and Swift

FitFlow Technical Needs

For the FitFlow redesign, the frontend technology should support iOS, Android and web with a consistent user experience. It also needs smooth workout screens, camera-based nutrition logging, real-time social features, notifications, secure authentication and AI-based recommendations. The case study already selected React Native because the team needed fast cross-platform development and good support for complex fitness interactions, so I used that as an important reference point when comparing the options.

Frontend Technology Comparison

Criteria	Flutter	React Native	Kotlin Multiplatform	Swift / SwiftUI
Development speed	Fast. Hot reload and many ready-made widgets.	Very fast, especially for teams that know JavaScript/TypeScript.	Medium. Shared code helps, but iOS and Android setup can take more work.	Fast for iOS only, but a separate Android/web solution is still needed.
Code reusability	High. One Dart codebase can cover mobile and web.	High for iOS/Android. Logic and some UI can also be shared with web.	High for shared business logic; UI sharing depends on the approach used.	Low for FitFlow because Swift mainly targets Apple platforms.
Performance	Very good and close to native for most screens and animations.	Very good for normal fitness app screens. Native modules can be used for heavy work.	Excellent because shared Kotlin can work closely with native platform code.	Excellent native performance on Apple devices.
Ecosystem support	Large package ecosystem and strong Google support.	Very large JavaScript ecosystem and many mobile libraries.	Growing ecosystem, but smaller than React Native and Flutter.	Very strong Apple ecosystem, but only for Apple platforms.
Learning curve	Requires learning Dart and Flutter widgets.	Easy if the team already knows React, JavaScript or TypeScript.	Requires Kotlin plus some knowledge of iOS tooling and platform code.	Requires Swift and Apple development experience.
Web compatibility	Good for app-like web experiences, but not ideal for every text-heavy web site.	Good when used with React or React Native Web, although some mobile components need web-specific changes.	Web support is improving, but the team may still need platform-specific work.	No direct Android/web reuse. A separate web frontend is needed.
AI/ML integration	Can call cloud AI APIs and use TensorFlow Lite through packages/native code.	Can call AI services easily and can connect to TensorFlow Lite or ML Kit through native modules.	Very good access to native ML libraries and platform APIs.	Excellent access to Core ML and Apple frameworks, but only on Apple devices.
Real-time features	Good with Firebase, WebSockets and other real-time libraries.	Very good with WebSockets, Firebase and JavaScript real-time libraries.	Good, but more platform integration may be required.	Good on Apple platforms using native frameworks.
Maintenance cost	Usually low because most code is shared.	Low to medium. Shared code saves time, but library updates must be managed carefully.	Medium because some platform-specific code and skills are still needed.	High for FitFlow because Android and web still need separate teams/codebases.

Criteria	Flutter	React Native	Kotlin Multiplatform	Swift / SwiftUI
Security	Good when secure storage, HTTPS and native security APIs are used correctly.	Good. Can use Keychain/Keystore, secure storage and native security libraries.	Very good because security-sensitive parts can use native APIs directly.	Very good on Apple platforms using Keychain, Secure Enclave and native controls.

Strengths and Weaknesses Summary

- Flutter: strong single-codebase development and very consistent UI. The main weakness is that the team must learn Dart and some web pages may need a different approach.
- React Native: very good balance between development speed, code sharing, ecosystem support and access to native features. The main weakness is that complex native features may still need iOS/Android-specific code.
- Kotlin Multiplatform: gives excellent native performance and flexible code sharing. The weakness is higher setup complexity and the need for more platform knowledge.
- Swift/SwiftUI: best choice for a pure Apple app, but not a good main choice for FitFlow because Android and web are also required.

Recommendation for FitFlow

I would select React Native as the main mobile frontend and use React for the web application. Shared TypeScript models, API services, validation logic and design components can be reused where practical. For performance-heavy features such as camera processing, health-device access or on-device AI, small native modules can be added when required.

This is the most practical choice for a mid-sized startup because it keeps development fast without giving up the ability to use native iOS/Android features. Flutter is a very close second choice, but React Native matches the technology direction already described in the FitFlow case study and fits well with a Node.js/TypeScript backend.

Activity 02

Compare Backend, Database and Authentication Options

Backend Framework Comparison

Criteria	Node.js / NestJS	Python / FastAPI	Go
Language / style	TypeScript, structured framework on Node.js.	Python, lightweight API framework.	Compiled Go language with simple services.
Development speed	High. Modules, validation, guards and dependency injection are already organised.	Very high for APIs and AI-related endpoints.	Medium. Simple language, but more code may be written manually.
Performance	Good for API and I/O-heavy workloads.	Good, especially with async endpoints.	Excellent performance and low memory usage.
Scalability	Good. Stateless services can be replicated easily.	Good. AI service can scale independently.	Excellent for high-concurrency services.

Criteria	Node.js / NestJS	Python / FastAPI	Go
Real-time features	Excellent WebSocket support and a good fit for social updates.	Good WebSocket support.	Good, but usually needs more manual setup.
AI/ML integration	Can call Python AI services, but JavaScript has fewer ML libraries than Python.	Excellent because most AI/ML tools already use Python.	Weak to medium for ML because the AI ecosystem is smaller.
Security / maintenance	Strong structure for guards, validation, rate limiting and testing. Easy for a TypeScript team to maintain.	Good security when configured correctly. Very clear API documentation.	Strong and simple runtime, but the team needs Go experience.
Best FitFlow use	Main application API and real-time social services.	AI microservice for personalisation and nutrition recognition.	Optional high-performance service if the product grows very large.

Database Comparison

Criteria	PostgreSQL	MongoDB	Firebase Firestore	DynamoDB
Data model	Relational tables with strong relationships and transactions.	Flexible JSON-like documents.	Cloud document database with built-in real-time sync.	Managed key-value/document database.
Query performance	Excellent for structured queries, joins and reporting when indexed properly.	Good for document queries, but complex relationships can be harder.	Fast for supported query patterns, but complex reporting is limited.	Very fast at large scale when access patterns are designed in advance.
Scalability	Scales well vertically and horizontally with managed cloud options/read replicas.	Good horizontal scaling.	Automatic scaling is a major strength.	Excellent automatic scaling.
Real-time	Usually added through backend WebSockets or change streams.	Can support change streams.	Excellent built-in real-time listeners.	Usually needs extra AWS services for real-time updates.
Health data handling	Very suitable for profiles, plans, nutrition records, permissions and audit logs because data integrity is strong.	Useful for flexible activity data, but relationships and consistency need more care.	Easy for mobile apps, but security rules and data structure must be designed carefully.	Strong security features, but more cloud-specific design knowledge is needed.
Cost / maintenance	Predictable for a mid-sized app when using a managed PostgreSQL service.	Reasonable, but operational cost depends on hosting.	Easy to start, but read/write based cost can rise with heavy real-time use.	Good at scale, but cost can be harder to estimate without known access patterns.

Authentication and Authorization Comparison

Criteria	Firebase Auth	AWS Cognito	Auth0	Supabase Auth
Setup speed	Very fast for mobile and web.	Medium; more AWS configuration is required.	Fast, with very good SDKs and dashboards.	Very fast when the project already uses Supabase/PostgreSQL.
Security features	Email/social login, MFA options, token-based auth.	Strong MFA, federation and AWS integration.	Strong MFA, RBAC/permissions and enterprise identity options.	Common login methods, JWTs and row-level security integration.
Scalability	Very good.	Excellent for very large user bases.	Excellent managed service.	Good for startup and mid-sized systems.
Compliance support	Depends on selected Google Cloud/Firebase services, plan, contracts and configuration.	Strong AWS compliance options, but the team must configure them correctly.	Strong compliance and enterprise support; HIPAA support is available only under qualifying plans/agreements.	Depends on plan, hosting region and how data is configured.
Cost	Low to start and simple for small/medium usage.	Can be cost-effective, but pricing/configuration is complex.	Higher cost as advanced/enterprise features are added.	Low to start and simple pricing.
Main weakness	Vendor lock-in and some advanced rules can become complex.	More difficult learning curve and AWS-specific setup.	Cost can become high for a growing consumer app.	Smaller identity ecosystem than Auth0/AWS/Firebase.

Recommended Combination

- Main backend: Node.js with NestJS. It gives a clean project structure, fast TypeScript development, REST APIs and strong WebSocket support for social features.
- AI service: Python with FastAPI. It is separated from the main backend so the AI team can use Python libraries and scale this service independently.
- Primary database: PostgreSQL. FitFlow has connected data such as users, workout plans, meal logs, subscriptions, groups and audit records, so a relational database gives better consistency and reporting.
- Caching and real-time layer: Redis plus NestJS WebSockets. Redis can reduce repeated database reads and help distribute real-time events between backend instances.
- Authentication: Auth0 for login, MFA and role-based access. It has strong security features and is easier to manage than building authentication ourselves. If cost becomes a major issue, Firebase Auth is a reasonable second option.

Security note: selecting a product that supports HIPAA or GDPR does not automatically make FitFlow compliant. The full system still needs proper consent, least-privilege access, encryption, secure logs, data retention/deletion rules, incident handling and the correct legal agreements. Also, FitFlow would only fall under HIPAA in specific healthcare situations, so this must be checked with the business and legal team instead of assuming every fitness app is automatically covered.

Activity 03

Prepare a Technology Comparison Matrix

I used a 1 to 5 score for each technology where 1 = poor and 5 = excellent. I did not put frontend frameworks, databases and authentication products into one direct ranking because they solve different problems. Instead, I used the same FitFlow priorities and created one weighted matrix for each technology layer. The final stack is made from the winner of each layer, with FastAPI added as a specialised AI service.

Frontend Weighted Decision Matrix

Criterion	Weight %	Flutter	React Native	Kotlin MP	Swift
Performance	20	5	4	5	5
Development speed	15	5	5	3	4
Code reusability	15	5	5	4	1
Web compatibility	10	3	4	3	1
Ecosystem	10	4	5	4	5
AI/ML integration	10	4	4	4	5
Security / native access	10	4	4	5	5
Maintenance cost	10	4	5	3	2
Weighted total / 5	100	4.40	4.50	3.95	3.55

Result: React Native has the highest weighted score (4.50/5). Flutter is very close, but React Native gives FitFlow a better match with the existing JavaScript/Node direction and a simple path to a React web frontend.

Backend Weighted Decision Matrix

Criterion	Weight %	NestJS	FastAPI	Go
Performance	20	4	4	5
Scalability	15	5	4	5
Development speed	15	5	5	3
Real-time support	10	5	4	4
AI/ML integration	10	3	5	2
Security	10	4	4	5
Maintainability	10	5	4	4
Cost efficiency	10	4	4	5

Criterion	Weight %	NestJS	FastAPI	Go
Weighted total / 5	100	4.40	4.25	4.20

Result: NestJS is the best main backend choice (4.40/5). FastAPI scores strongly because of AI support, so I would still use it as a separate AI microservice instead of forcing all AI code into Node.js.

Database Weighted Decision Matrix

Criterion	Weight %	PostgreSQL	MongoDB	Firestore	DynamoDB
Data integrity & queries	20	5	3	3	3
Scalability	15	5	4	5	5
Performance	15	5	4	4	5
Real-time support	10	3	4	5	4
Security	15	5	4	4	5
Cost	10	4	4	3	3
Maintainability	10	4	4	5	3
Flexibility	5	4	5	5	3
Weighted total / 5	100	4.55	3.85	4.10	4.00

Result: PostgreSQL has the highest score (4.55/5). Its main advantage for FitFlow is reliable structured data and strong querying for users, plans, meals, subscriptions and audit records. Real-time behaviour can be added at the service layer rather than making the database do everything.

Authentication Weighted Decision Matrix

Criterion	Weight %	Firebase Auth	AWS Cognito	Auth0	Supabase Auth
Security	20	4	5	5	4
Development speed	15	5	3	5	5
Cross-platform support	10	5	5	5	5
MFA / RBAC	15	4	5	5	4
Scalability	10	5	5	5	4
Compliance support	15	3	5	5	3
Cost	10	4	4	2	5
Maintainability	5	5	3	4	4
Weighted total / 5	100	4.25	4.50	4.65	4.20

Result: Auth0 receives the highest score (4.65/5) because FitFlow needs secure cross-platform login, MFA and manageable roles. Its main disadvantage is cost, so this decision should be reviewed again when the real user count and budget are known.

Recommended Technology Stack

Layer	Selected Technology	Reason
Mobile frontend	React Native + TypeScript	Fast cross-platform development for iOS and Android.
Web frontend	React + shared TypeScript modules	Keeps the web experience flexible while reusing logic and API code.
Main backend	Node.js + NestJS	Structured APIs, WebSockets and good maintainability for a mid-sized team.
AI service	Python + FastAPI	Best fit for recommendation models and nutrition image analysis.
Database	PostgreSQL	Reliable relational data, transactions, reporting and audit records.
Cache / real-time	Redis + WebSockets	Fast repeated reads and real-time social/workout events.
Authentication	Auth0	Managed login, MFA, role-based access and good cross-platform SDK support.
On-device AI	TensorFlow Lite / ML Kit where practical	Reduces latency and unnecessary transfer of sensitive images/data.

Activity 04

Design a High-Level Architecture

The following architecture separates the mobile/web user interface, the main business API, the AI workload and the data layer. This makes the system easier to maintain and allows the AI service or real-time features to scale without rebuilding the full application.

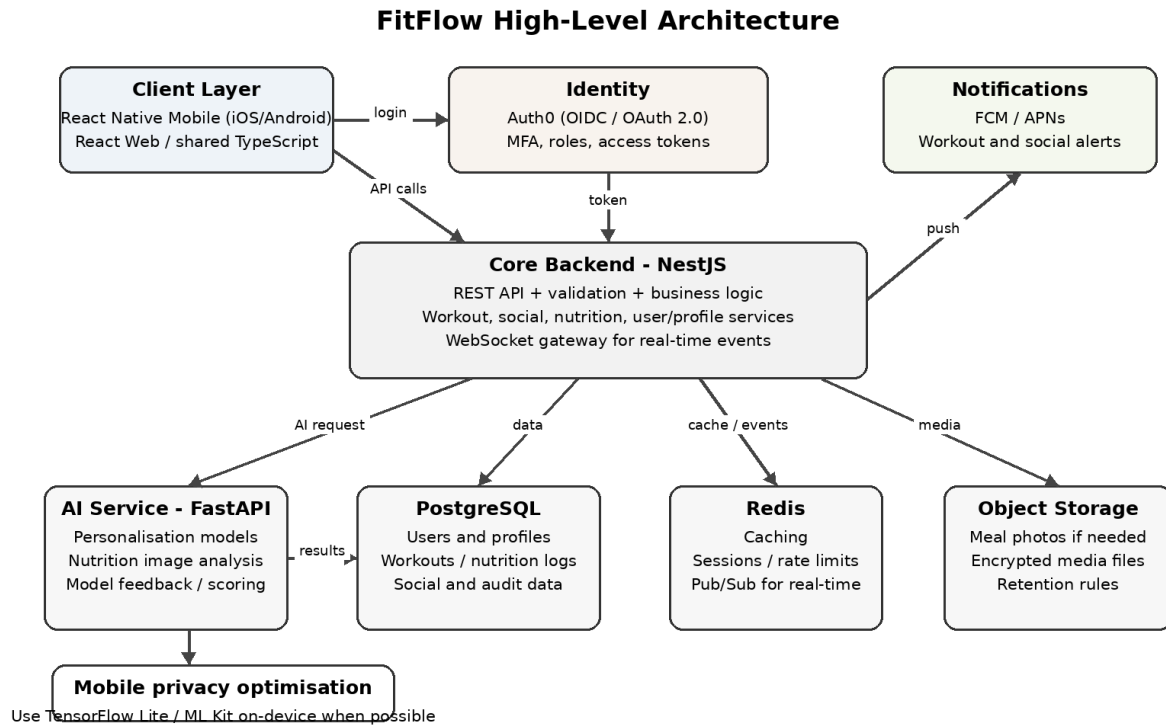


Figure 1: Proposed high-level architecture for FitFlow

Key Components

Component	Purpose
React Native mobile app	iOS/Android user interface for onboarding, workouts, social features, progress and camera-based meal logging.
React web app	Web access for the same account using shared TypeScript models and API services where possible.
Auth0	Handles sign-up, login, MFA, token issuing and role/permission management.
NestJS backend	Main REST API and business logic for users, plans, social features, nutrition records and notifications.
WebSocket + Redis layer	Sends live challenge/social updates and reduces repeated database work through caching.
FastAPI AI service	Runs personalised workout logic, nutrition image analysis and model scoring.
PostgreSQL	Stores structured application data, relationships, subscriptions and audit history.
Object storage	Stores meal images or other media only when needed, with encryption and retention controls.
FCM / APNs	Sends workout reminders and social notifications to mobile devices.

Critical Data Flows

- Personalised workout plan: user signs in -> mobile/web sends profile and schedule request -> NestJS reads recent workout data -> FastAPI AI service produces a recommendation -> NestJS stores the accepted plan in PostgreSQL -> the plan is returned to the user. The user can change or reject the suggestion, and that feedback can be saved for future recommendations.

- Social sharing: user posts a progress update or joins a challenge -> NestJS checks the access token and privacy settings -> data is saved in PostgreSQL -> an event is sent through the WebSocket/Redis layer -> approved friends or group members receive the update in real time.
- Nutrition tracking: user takes a meal photo -> the app first uses on-device processing where possible -> if cloud analysis is required, only the needed image/data is sent securely to the AI service -> the prediction is shown to the user for confirmation -> confirmed nutrition data is stored in PostgreSQL. The app shows a clear success message after saving.

Security Considerations

- Use HTTPS/TLS for all network traffic and do not send passwords to the FitFlow backend.
- Use short-lived access tokens, MFA for risky logins and role-based access for admin/support functions.
- Store mobile tokens in Keychain/Keystore or another secure storage method, not in plain local storage.
- Encrypt databases, backups and stored media. Keep sensitive profile data separate from public social content.
- Apply input validation, rate limiting, secure file upload checks and audit logging on the backend.
- Collect only the health/wellness data that is needed. Provide clear consent, export/delete options and retention rules for GDPR-style privacy requirements.

Scalability and Integration Considerations

- Keep NestJS services stateless so multiple backend instances can run behind a load balancer.
- Scale the FastAPI AI service separately because AI requests may need more CPU/GPU resources than normal API calls.
- Use Redis for caching popular dashboard data and for sharing WebSocket events between backend instances.
- Add indexes to PostgreSQL for user, date, workout and social queries. Read replicas can be added later if reporting traffic becomes heavy.
- Use REST/JSON for normal client requests and WebSockets only for features that actually need real-time updates. This keeps the system simpler.
- Use monitoring, error tracking and audit logs so the team can quickly find failures after deployment.

Architecture Decision Record (ADR-001)

ADR Item	Decision / Note
Title	Technology stack for FitFlow redesign
Status	Accepted for the redesign prototype / first production version
Context	FitFlow needs iOS, Android and web support, AI personalisation, camera nutrition logging, real-time social features, strong privacy and fast delivery by a mid-sized team.
Decision	Use React Native for mobile, React for web, NestJS for the main backend, FastAPI for AI, PostgreSQL for core data, Redis/WebSockets for real-time features, Auth0 for identity, and on-device ML where practical.
Main reasons	Good code reuse, strong ecosystem, easy team maintenance, separate scaling of AI, reliable relational data and managed authentication.
Trade-offs	More than one backend technology is used, Auth0 can become expensive, and some native mobile code may still be required for camera/health/AI features.

ADR Item	Decision / Note
Alternatives considered	Flutter for a single UI framework, Firebase-first backend for faster setup, Kotlin Multiplatform for more native control, and Go for very high backend performance.
Review point	Review cost, performance and user growth after the first production release. Replace components only if real measurements show a problem.

Activity 05

Create a GitHub Repository

Repository Details

Repository name: fitflow-redesign

Repository link: <https://github.com/LakshanWAKTK-IT23698918/fitflow-redesign>

Initial Project Folder Structure

fitflow-redesign/

```

|
├── frontend/
|   └── README.md
|
├── backend/
|   └── README.md
|
├── ai-service/
|   └── README.md
|
├── docs/
|   ├── architecture.md
|   ├── comparison-matrix.md
|   └── tech-stack.md
|
├── .gitignore
└── README.md

```

README.md Content

The README should be short and useful so another team member can understand the project quickly. I would include the following information:

- Project name and purpose: FitFlow Redesign - a cross-platform fitness application redesign focused on personalisation, easier nutrition tracking and positive social accountability.

- Main features: AI-generated workout plans, workout editing, progress dashboard, camera-based nutrition logging, private social challenges and notifications.
- Technology stack: React Native, React, NestJS, FastAPI, PostgreSQL, Redis, Auth0 and TensorFlow Lite/ML Kit where needed.
- Repository structure: short explanation of frontend, backend, ai-service and docs folders.
- Local setup: required Node.js, Python and PostgreSQL versions can be added when implementation starts, followed by install/run commands.
- Security note: secrets must be stored in environment variables and .env files must never be committed to GitHub.
- Links to docs: comparison matrix, architecture diagram and ADR.

Tech Stack Summary File

Layer	Technology
Mobile	React Native + TypeScript
Web	React + TypeScript
Backend	Node.js + NestJS
AI	Python + FastAPI
Database	PostgreSQL
Cache / Real-time	Redis + WebSockets
Authentication	Auth0
On-device AI	TensorFlow Lite / ML Kit

.gitignore Essentials

```
node_modules/
.env
.env.*
!.env.example
dist/
build/
.expo/
__pycache__/
.venv/
*.pyc
.DS_Store
.idea/
.vscode/
```

Basic Repository Settings

- Use main as the protected default branch.
- Require changes to go through a pull request before merging to main.
- Require at least one review for important changes when working as a team.
- Block force-pushes to main so commit history cannot be easily overwritten.
- Enable GitHub secret scanning/dependency alerts where available and never commit API keys, database passwords or Auth0 secrets.

- Optional CI/CD: run frontend/backend lint and tests plus Python tests automatically on each pull request.

First Commit

After creating the repository and adding the folders and documents above, I would make the first commit with the supporting work from Activities 1-4.

```
git init
git add .
git commit -m "Initial FitFlow redesign setup"
git branch -M main
git remote add origin https://github.com/rakiiith/fitflow-redisgn.git
git push -u origin main
```

GitHub repository link - <https://github.com/LakshanWAKTK-IT23698918/fitflow-redesign>